Documentation for `geo.chpl`

Nelson Luís Dias (`nelsonluisdias@gmail.com`)

February 24, 2026

Abstract

`geo` is a module for things that vary spatially

Part I

Theory

A vector (u, v) can be in either of 4 quadrants. In this module, the quadrants are 0,1,2,3 (instead of 1,2,3,4). In procedure `inxy` below, we compute consecutive vectors from a fixed point (x_p, y_p) to the points (x_b, y_b) along a closed boundary. For each consecutive pair, there is an increment that determines the difference in quadrants between the first and second vectors.

These increments can be subsumed in a "from/to" matrix. The "from" is the quadrant of the first vector, and is listed along the lines i of the matrix; the "to" is the quadrant of the second vector, and is listed along the columns j. Most, but not all, of the matrix's elements (i, j) are equal to $j - i$. In position $(0, 3)$, $+3$ is replaced by -1, because when the first vector is in quadrant 0 and the second vector is in quadrant 3, there is a "-1" rotation; by the same token, when the first vector is in quadrant 3 and the second vector is in quadrant 0, there is a "$+1$" rotation. In quadrant units, a rotation of $+1$ is equivalent to a rotation of $\pi/2$ radians.

In `inxy`, we sum the increments as (x_b, y_b) wanders around the boundary. If (x_p, y_p) is outside of the boundary, the sum will be zero; otherwise, it will be ± 4.

Table 1: The skew-symmetric matrix of quadrant increments $j - i$. In position $0, 3$, $+3$ is replaced by -1, and in position $3, 0$, -3 is replaced by $+1$

from i \ to j	0	1	2	3
0	0	$+1$	$+2$	-1
1	-1	0	$+1$	$+2$
2	-2	-1	0	$+1$
3	$+1$	-2	-1	0

Part II

Constants and procedures

1 Constants

There are no constants yet.

2 `findquad`: in which quadrant is a point/vector?

2.1 Use

`findquad`: finds the quadrant to which the point/vector (delx,dely) belongs.

```
private proc findquad(
   const in delx: real,       // abscissa of point
   const in dely: real        // ordinate of point
   ): int {                   // quadrant of point (0, 1, 2, 3)
```

2.2 Theory

`findquad` is private and should not be called from outside of `geo.chpl`. It is fairly self-explanatory, so check the source code.

3 `inxy`: is the point (xp,yp) inside perimeter (axboun,ayboun)?

3.1 Use

`inxy`: is the point (xp,yp) inside perimeter (axboun,ayboun)?

```
proc inxy(
   const in xp: real,              // is this point in or out?
   const in yp: real,              // is this point in or out?
   const ref axboun: [] real,      // abscissas of perimeter
   const ref ayboun: [] real       // ordinates of perimeter
   ): bool where (axboun.rank == 1) && (ayboun.rank == 1) {
```

3.2 Theory

`inxy` uses the "Winding point algorithm" (see https://en.wikipedia.org/wiki/Point_in_polygon) and follows the ideas in Weiler (1994), but my implementation is considerably simpler than Weiler's (implemented in C).

Given the point $P = (x_p, y_p)$, `inxy` draws sucessive vectors

$$(\delta_{xi}, \delta_{yi}) = (x_{bi} - x_p, y_{bi} - y_p)$$

from P to each point "i" (x_{bi}, y_{bi}) in the boundary, for $i = 0, \ldots, n$. We require that

$$x_{bn} = x_{b0},$$
$$y_{bn} = y_{b0},$$

so that the boundary "closes". We define the "rotation" from $(\delta_{xi-1}, \delta_{yi-1})$ to $(\delta_{xi}, \delta_{yi})$ by the quadrant increments of table 1, and sum the rotations `delquad` from 1 to n. If P is an interior point, the total sum of the rotations should be ± 4; otherwise, it is 0.

References

Weiler, K. (1994). An Incremental Angle Point in Polygon Test. In Heckbert, P. S., editor, *Graphics Gems IV*, chapter I.3, pages 16–23. Morgan Kaufmann/Academic Press, San Diego, San Francisco, New York, Boston, London, Sydney, Tokyo.